

UNIwersYTET VIZJA • SCHOOL OF COMPUTER SCIENCE & TECHNOLOGIES

# Design and Implementation of a Web-Based Student Management System for Educational Institutions

LIVE [diploma-project-sms-management-system.onrender.com](https://diploma-project-sms-management-system.onrender.com)

## STUDENT

**Maksym Shpak**

Index: [nr albumu]

## SUPERVISOR

**[dr / promotor]**

Diploma seminar

## FIELD OF STUDY

**Computer Science**

Engineering degree • 2026

# Presentation Agenda

- |                                                                                                          |                                                                                                |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <p><b>01 Problem Statement &amp; Purpose</b><br/>Spreadsheets, paper registers, no audit trail</p> <hr/> | <p><b>06 Key Features</b><br/>Gradebook, attendance, analytics, CSV export</p> <hr/>           |
| <p><b>02 Comparative Analysis</b><br/>Moodle, USOS, Google Classroom — and the gap</p> <hr/>             | <p><b>07 Role-Based Access Control</b><br/>Administrator and teacher data isolation</p> <hr/>  |
| <p><b>03 Technology Stack</b><br/>Django 6, Python 3.12, PostgreSQL 16</p> <hr/>                         | <p><b>08 Internationalisation</b><br/>269 translatable strings, English and Polish</p> <hr/>   |
| <p><b>04 System Architecture</b><br/>Browser › Render TLS › Gunicorn › PostgreSQL</p> <hr/>              | <p><b>09 Quality Assurance &amp; CI</b><br/>50 automated tests, four CI gates</p> <hr/>        |
| <p><b>05 Database Design</b><br/>6 models, 5 migrations, constraints in the schema</p> <hr/>             | <p><b>10 Deployment &amp; Results</b><br/>Live on Render + Neon at zero hosting cost</p> <hr/> |

# Problem Statement & Purpose

## 4–6 h

per week per teacher spent on manual record-keeping

## 3 places

a single student record is split across today

## 0

audit trail of who entered or edited a grade

### THE PROBLEM

- **Fragmented records** — spreadsheets, paper registers, email threads
- **No single view** — grades and attendance never meet in one place
- **Manual arithmetic** — averages recalculated by hand, errors slip through
- **No accountability** — nobody can prove who recorded a mark
- **No confidentiality** — every teacher can see every student
- **No Polish interface** — staff work in English-only tools

### THE PURPOSE

- **One source of truth** — a relational schema with enforced constraints
- **Instant analytics** — KPI tiles and three charts from ORM aggregation
- **Automatic authorship** — every grade stores who assigned it
- **Scoped visibility** — teachers see their own courses, admins see all
- **Bilingual by design** — the whole interface switches EN ⇌ PL
- **Deployable** — one build script, HTTPS, managed PostgreSQL

**Result:** one auditable, role-aware platform now managing 25 students, 8 courses, 87 enrollments and 53 grades in production.

# Comparative Analysis

| CRITERION           | MOODLE                  | USOS                   | GOOGLE CLASSROOM     | SMS — THIS PROJECT                 |
|---------------------|-------------------------|------------------------|----------------------|------------------------------------|
| Target user         | Large universities      | Polish public HEIs     | Schools and courses  | Departments and small institutions |
| Deployment          | Self-hosted, heavy      | Vendor-managed         | Google cloud only    | Self-hosted, one Django app        |
| Setup effort        | Days of plugin work     | Institutional contract | Minutes, no control  | Minutes — a single build script    |
| Gradebook           | Yes, complex            | Yes                    | Basic                | Yes, with author audit trail       |
| Attendance register | Plugin required         | Yes                    | Not available        | Built in, four statuses            |
| Role isolation      | Configurable, intricate | Yes                    | Limited              | Enforced at queryset level         |
| Bilingual EN / PL   | Language packs          | Polish-first           | Yes                  | Native, 269 strings                |
| Analytics           | Reports plugin          | Institutional reports  | Minimal              | Chart.js dashboard + CSV           |
| Cost of ownership   | GPL + hosting           | Licensed               | Free, data in Google | Open source, €0 hosting            |

**Positioning:** the established platforms are either too heavy or too closed for a single department; SMS trades breadth for a focused, auditable and fully self-hosted core.

# Technology Stack

## BACKEND

Runtime · data · authentication

- Python 3.12
- Django 6.0.6
- PostgreSQL 16 (Neon)
- SQLite 3 for development
- Django session authentication
- Custom user model with roles
- Django ORM · 5 migrations
- dj-database-url 2.1

## FRONTEND

Templates · charts · theming

- Django Templates · 28 files
- Semantic HTML5
- CSS custom properties · 1 771 lines
- Vanilla JavaScript ES6 · 399 lines
- Chart.js 4 — doughnut and bars
- Turbo 8 page transitions
- Bootstrap Icons
- Light and dark themes

## INFRASTRUCTURE

Hosting · operations · CI

- Render web service
- Neon managed PostgreSQL
- Gunicorn 22 (WSGI)
- WhiteNoise 6.6 static files
- HTTPS with HSTS preload
- GitHub Actions CI
- Git version control
- Zero CDN dependencies

**No build step, no separate API server.** Every asset is vendored locally, so the application runs and demonstrates fully offline.

# System Architecture

## CLIENT

### Browser

HTML · CSS · ES6

Turbo navigation

## EDGE

### Render · TLS 443

HTTPS, HSTS

static via WhiteNoise

## APPLICATION

### Gunicorn + Django 6

URL › View › Template

ORM aggregation

## DATA

### PostgreSQL 16

Neon managed

daily backups

## REQUEST PIPELINE — MIDDLEWARE

Security

Session

CSRF

Locale

WhiteNoise

Authentication

Role decorators

Queryset scoping

## SUPPORTING SERVICES

### SMTP email

Password reset links

### GitHub Actions

Tests and deploy gates

### Django Admin

Superuser operations

### CSV export

Role-scoped reporting

### /api/dashboard/

JSON metrics endpoint

# Database Design

## User *extends AbstractUser*

id  
username • email  
first\_name • last\_name  
role → admin | teacher  
password (PBKDF2)  
is\_active

## Student *academic identity*

id  
student\_number (unique)  
first\_name • last\_name  
email  
study\_program  
date\_enrolled • is\_active

## Course *taught unit*

id  
course\_code (unique)  
course\_name • description  
semester • credits  
max\_students  
teacher\_id → User

## Enrollment *student ↔ course*

id  
student\_id → Student  
course\_id → Course  
enrollment\_date  
status → active | completed | dropped  
unique (student, course)

## Grade *assessment*

id  
enrollment\_id → Enrollment  
grade\_value 2.0 – 5.0  
letter\_grade (derived)  
assigned\_by → User  
date\_assigned • comments

## Attendance *daily register*

id  
enrollment\_id → Enrollment  
date  
status → present | absent | late | excused  
notes  
unique (enrollment, date)

**Integrity where it belongs:** unique constraints, foreign keys and cascade rules live in the schema, so no interface path can create a duplicate enrollment or an orphaned grade.

# Key Features

## Academic Records

Complete CRUD for students, courses and enrollments. Search, programme and status filters plus pagination — and the filters survive every page change because they are carried in the query string.

## Gradebook with Audit Trail

Numeric grades from 2.0 to 5.0 map automatically to letter grades. Each record stores assigned\_by, so it is always provable which teacher entered or last edited a mark.

## Attendance Register

Present, absent, late and excused, with one row per enrollment per day guaranteed by a database constraint. The resulting attendance rate feeds the analytics directly.

## Analytics Dashboard

Four KPI tiles and three Chart.js visualisations — grade distribution, enrollments per course, students per programme — all computed by ORM aggregation and also served as JSON at `/api/dashboard/`.

## Capacity & Validation

Enrollment is refused once a course reaches `max_students`. Duplicate enrollments and duplicate attendance rows are impossible by design, not by convention.

## Export & Password Recovery

Students and grades export to CSV with translated headers, scoped to the caller's role. Password reset delivers a signed, single-use link over SMTP.



# Role-Based Access Control

| CAPABILITY   | ADMINISTRATOR               | TEACHER                |
|--------------|-----------------------------|------------------------|
| Dashboard    | Institution-wide metrics    | Own courses only       |
| Students     | Create, edit, delete        | Read-only, own courses |
| Courses      | Full CRUD, assigns teachers | Read-only, own courses |
| Enrollments  | Full CRUD                   | Read-only, own courses |
| Grades       | Full CRUD, all courses      | Full CRUD, own courses |
| Attendance   | Full CRUD, all courses      | Full CRUD, own courses |
| CSV export   | Complete dataset            | Scoped to own courses  |
| Django Admin | Available                   | Not available          |

## Guarded twice

@admin\_required and @teacher\_or\_admin\_required protect the route; the queryset is filtered again inside the view. A guessed URL returns nothing.

## IDOR-proof

Every detail, edit and delete view re-checks object ownership before it touches the database.

## Proven by tests

TeacherDataIsolationTests asserts that a teacher can neither read nor modify records belonging to a course they do not own.

**Design decision:** two roles instead of many. Enough to demonstrate real authorisation and data confidentiality, simple enough to stay verifiable.

# Internationalisation

269

translatable  
strings

2

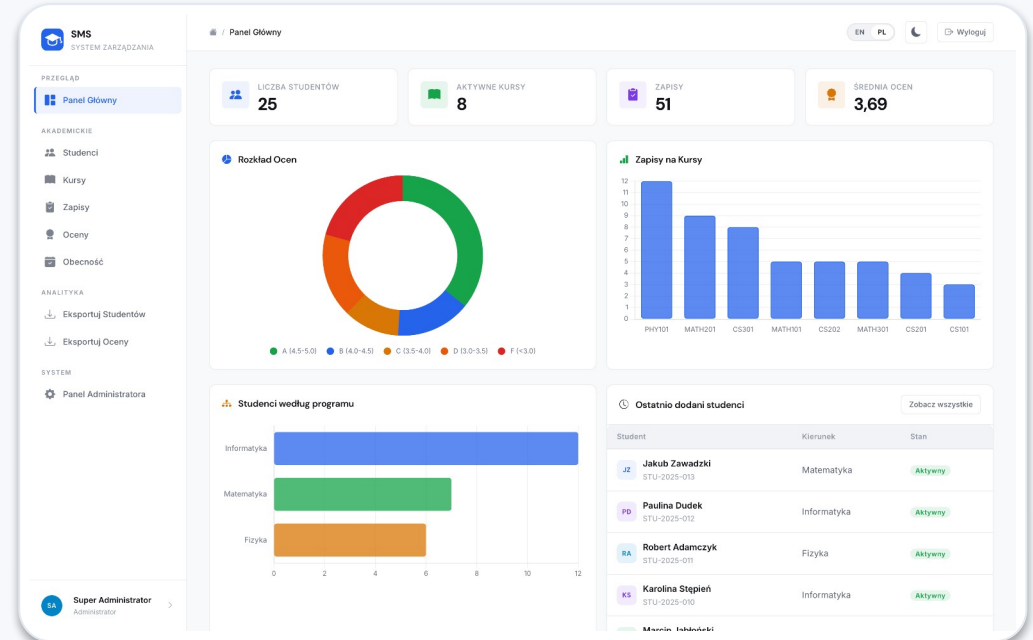
interface  
languages

0

hardcoded  
UI strings

## HOW IT IS DONE

- **Templates** — every string in `{% trans %}` or `{% blocktrans %}`
- **At the source** — model choices, placeholders, flash messages
- **Dates** — locale formats: 26/08/2025 or 26.08.2025
- **Catalogue** — stored programmes and courses via a `trans_db` filter
- **Numbers** — unlocalised for JavaScript, then `Intl.NumberFormat`
- **Continuity** — the switch keeps the page and persists per session



The dashboard in Polish — labels, catalogue values and number formats all follow the active locale.

**Polish is not a bolt-on:** the catalogue is compiled during the build, committed as `django.mo`, and guarded by `LocalizationTests`.

# Quality Assurance & CI

**50**

automated tests

**14**

test classes

**4**

CI gates per push

**0**

known failing tests

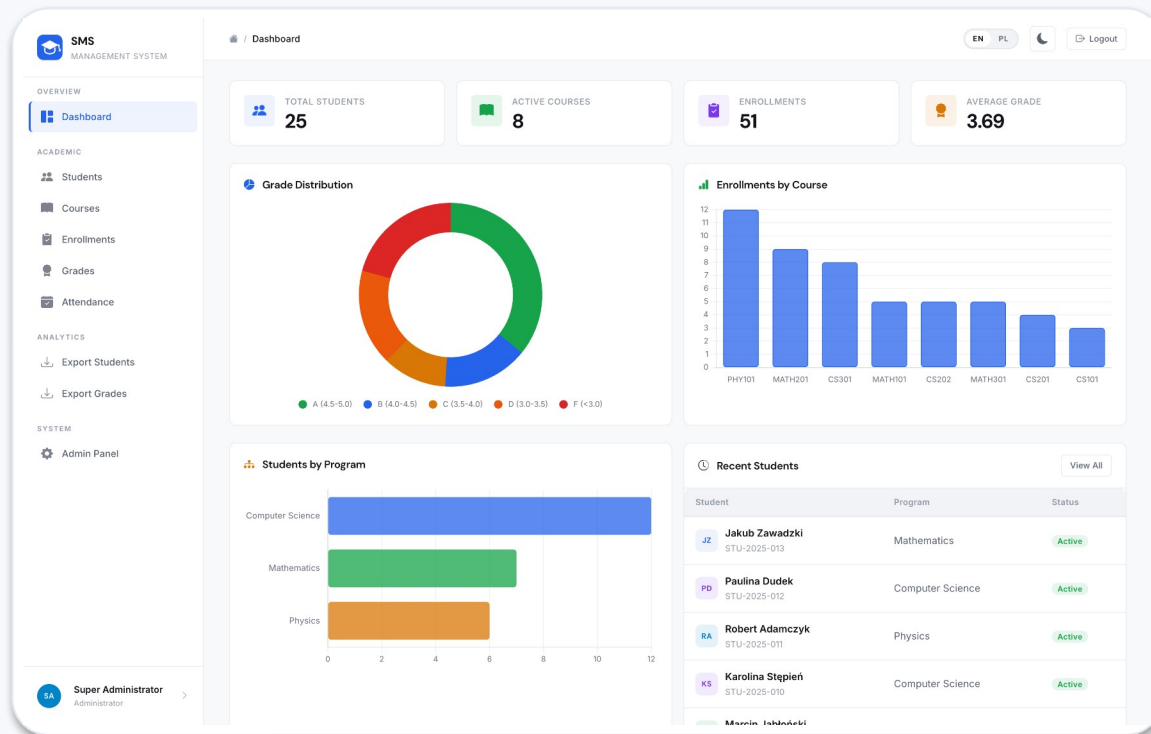
## TEST SUITE

**SmokeRenderTests** pages render for both roles**CrudFlowTests** create, edit, delete end to end**EnrollmentCapacityTests** max\_students enforced**PaginationTests** filters survive paging**DashboardApiTests** payload differs per role**DeploymentSeedTests** seeding is idempotent**TeacherDataIsolationTests** scoping and IDOR**GradeAuditTests** assigned\_by always recorded**LocalizationTests** EN  $\rightleftharpoons$  PL, database values**LogoutSafetyTests** logout is POST-only**PasswordResetEmailTests** reset mail is signed**CoreModelTests** letter grades and invariants

## CI PIPELINE

- **git push** GitHub Actions starts
- **manage.py test** the full suite must pass
- **makemigrations --check** no model drift allowed
- **check --deploy** production security audit
- **collectstatic** assets build cleanly
- **build.sh on Render** migrate, compile PL, seed, serve

# Interface — Analytics



Administrator dashboard — four KPI tiles and three Chart.js views



The same dashboard in dark theme, driven by CSS custom properties

# Interface — Records

**Students**  
Manage student records and academic information

Search students... All Programs All Status Filter

| Student             | Student Number | Program          | Enrolled   | Courses | Avg. Grade | Status | Actions |
|---------------------|----------------|------------------|------------|---------|------------|--------|---------|
| Robert Adamczyk     | STU-2025-011   | Physics          | 08/26/2025 | 3       | 3.00       | Active |         |
| Paulina Dudek       | STU-2025-012   | Computer Science | 01/10/2025 | 2       | 3.87       | Active |         |
| Agnieszka Dąbrowska | STU-2024-010   | Mathematics      | 10/04/2024 | 1       | 3.00       | Active |         |
| Natalia Grabowska   | STU-2025-004   | Physics          | 12/28/2024 | 3       | 3.00       | Active |         |
| Marcin Jabłoński    | STU-2025-009   | Mathematics      | 03/23/2025 | 1       | 2.75       | Active |         |
| Monika Jankowska    | STU-2024-012   | Physics          | 01/18/2025 | 2       | 4.00       | Active |         |
| Piotr Kamiński      | STU-2024-005   | Computer Science | 11/09/2024 | 2       | 3.50       | Active |         |
| Alexander Kowalski  | STU-2024-001   | Computer Science | 09/26/2024 | 4       | 3.60       | Active |         |
| Krzysztof Kozłowski | STU-2024-011   | Computer Science | 10/14/2024 | 2       | 2.50       | Active |         |
| Ewa Krawczyk        | STU-2025-002   | Mathematics      | 07/27/2025 | 0       | 3.25       | Active |         |
| Daniel Król         |                |                  |            |         |            |        |         |

Student register — search, programme and status filters, pagination

**Alexander Kowalski**  
# STU-2024-001 | Email: a.kowalski@student.edu | Phone: +48 512 345 678 | Active

**Student Information**

STUDY PROGRAM: Computer Science  
DATE ENROLLED: Sept. 26, 2024  
EMAIL: a.kowalski@student.edu  
PHONE: +48 512 345 678  
DATE OF BIRTH: March 27, 2001

ADDRESS: ul. Akademicka 100, Warsaw, Poland  
TOTAL COURSES: 5  
AVERAGE GRADE: 3.60

**Enrolled Courses & Grades**

| Course                                | Semester | Credits | Status    | Grade   | Date       |
|---------------------------------------|----------|---------|-----------|---------|------------|
| MATH101<br>Calculus I                 | Fall     | 6       | Active    | 3.5 (C) | 06/18/2026 |
| CS202<br>Data Structures & Algorithms | Spring   | 6       | Completed | 4.5 (A) | 06/18/2026 |
| PHY101<br>Physics I — Mechanics       | Fall     | 6       | Active    | 4.5 (A) | 06/18/2026 |
| MATH201<br>Linear Algebra             | Spring   | 5       | Active    | 3.0 (D) | 06/18/2026 |
| MATH301<br>Statistics & Probability   | Summer   | 4       | Active    | 2.5 (F) | 06/18/2026 |

Student profile — enrollments, grades and attendance in one view

**Grade Distribution**

TOTAL STUDENTS: 25  
ACTIVE COURSES: 8  
ENROLLMENTS: 51  
AVERAGE GRADE: 3.69

Grade Distribution: A (4.5-5.0), B (4.0-4.5), C (3.5-4.0), D (3.0-3.5), F (<3.0)

Responsive  
down to 430 px

**Every table is role-aware:** a teacher opening the same page sees only the students enrolled in the courses they teach, and the action buttons disappear.

# Deployment & Live Application

LIVE <https://diploma-project-sms-management-system.onrender.com>

25

students

8

courses

87

enrollments

53

grades

€0

monthly cost

## DEPLOYMENT PIPELINE

- **Push to main** — GitHub Actions runs the suite and the deploy gates
- **Render build.sh** — installs dependencies, compiles the Polish catalogue
- **collectstatic** — WhiteNoise fingerprints and compresses every asset
- **migrate + seed\_demo** — schema applied, demo data only if empty
- **Gunicorn** — serves the WSGI application behind Render TLS

## PRODUCTION HARDENING

- **HTTPS everywhere** — redirect plus HSTS for one year, preload enabled
- **Cookies** — Secure and HttpOnly for session and CSRF
- **Headers** — nosniff, same-origin referrer, X-Frame-Options DENY
- **Secrets** — SECRET\_KEY, database URL and SMTP come from the environment
- **DEBUG off** — settings validated by manage.py check --deploy

# Results & Future Work

## DELIVERED

- **Live deployment** — HTTPS, managed PostgreSQL, automated builds
- **Six-model schema** — constraints and referential integrity in the database
- **Role isolation** — administrators and teachers see strictly their own data
- **Audited gradebook** — grade authorship recorded on every entry
- **Attendance register** — four statuses, one record per day per enrollment
- **Analytics** — KPI tiles, three charts, CSV export, JSON endpoint
- **Bilingual interface** — 269 strings, locale-aware dates and numbers
- **50 automated tests** — green in continuous integration
- **Offline-ready** — no CDN, every asset vendored in the repository

## FUTURE DEVELOPMENT

- **Student portal** — a third role with read-only access to own results
- **Semester transcripts** — generated PDF reports per student
- **Email notifications** — automatic alerts on new grades and absences
- **REST API** — a documented, token-authenticated public interface
- **Timetabling** — room and slot scheduling with conflict detection
- **LMS integration** — import and export against Moodle and USOS
- **Audit log** — a full change history beyond grade authorship
- **Two-factor login** — TOTP for administrator accounts
- **Progressive web app** — installable offline attendance taking

**Student Management System:** an engineering project that left the laptop — deployed, tested, localised, and ready to run in a real department.



# Thank you for your attention

Questions are welcome.

---

**Maksym Shpak**

Computer Science · Index: [nr albumu]

<https://diploma-project-sms-management-system.onrender.com>